Implementation of Dynamic Prompt Customization in ops/bin/open: Architectural Analysis and Compliance Report

1. Introduction: The Philosophy of Repo-Resident AI and Context Rehydration

The operational efficacy of a repository-resident Artificial Intelligence (AI) system is fundamentally constrained by the quality, precision, and "freshness" of the context provided at the genesis of an operational session. In the Stela ecosystem, the ops/bin/open binary serves as the "Timekeeper" and the "Gateway".[1] It is not merely a script that prints text to a terminal; it is the mechanism that anchors the stateless AI agent to a specific moment in the repository's history, defined by a Git commit hash, a branch state, and a governance contract. The architectural evolution of this component from a static string emitter to a dynamic, context-aware template engine is a critical maturity milestone required to reduce operator friction and enforce the rigorous standards of the **Policy of Truth (PoT.md)**.[1]

This report provides an exhaustive technical and governance analysis of implementing dynamic prompt customization within ops/bin/open. The proposed solution addresses the immediate operational requirement to support the workflow: **Run Open -> Run Dump -> Attach Artifacts -> Copy/Paste Prompt -> Insert Fields**. More importantly, it transforms the "Insert Fields" phase from a manual, error-prone data entry task into an automated, system-verified state injection process. By shifting the prompt payload from the executable binary (Run) to the documentation tree (Explain), the architecture aligns strictly with the Filing Doctrine, establishing a "Single Source of Truth" (SSOT) for operator intent.

The analysis proceeds by dissecting the current "Monolithic" implementation, identifying its governance violations and workflow bottlenecks, and then detailing a target architecture based on **Dynamic Prompt Injection**. This architecture relies on a formalized **Prompt Registry**, a **Variable Harvesting Subsystem**, and a **Safe Substitution Engine**, all implemented within the constraints of the existing Bash toolchain to maintain the lightweight, dependency-free nature of the Ops Kernel.[1]

1.1 The Role of ops/bin/open as the Timekeeper

In a stateless system, context is not memory; context is a file.[1] The ops/bin/open binary functions as the "Timekeeper" because it freezes the repository state at the moment of

session initiation. According to the system's technical specifications, it captures git rev-parse --abbrev-ref HEAD (Active Branch), git rev-parse --short HEAD (Commit Hash), and git status --porcelain (Dirty State).[1]

This functionality is critical because the AI agent—whether it is ChatGPT, Claude, or Gemini—has no inherent knowledge of "now." Without the artifact generated by ops/bin/open, the agent operates in a temporal vacuum, potentially hallucinating that it is working on a stable codebase when, in reality, the working tree is dirty, or referencing a file structure that existed three commits ago. The current implementation creates a OPEN-<tag>-<branch>-<hash>.txt artifact that serves as the "anchor" for the session.[1]

However, while the *physical* state (Git) is captured effectively, the *intentional* state (the specific role the AI plays, such as Auditor or Architect) is currently hardcoded and static. The prompt instructs the AI to adopt a "collaborator stance" or a "reviewer stance" based solely on the output format flag, which is a binary and insufficient distinction for complex operations.[1] To fully realize the "Timekeeper" philosophy, ops/bin/open must allow the operator to inject precise, version-controlled intent at the moment of session creation, ensuring that the "Why" is as strictly defined as the "What" and the "When."

1.2 The Imperative for Context Rehydration

"Context Rehydration" is the process of rebuilding the full operational awareness of an agent from zero. The documentation explicitly states: "If it is not in the Context, it does not exist".[1] Currently, the ops/bin/open binary provides a "Freshness Gate" and a set of "Canon Pointers," but it leaves the definition of the specific task largely blank.

The snippet [1] shows the current output of the tool:

- Active branch: main
- HEAD short hash: 77413dda4
- DP id/date (if applicable): (none)
- Intent for today: DP-OPS-0040
  While the intent is populated if the flag is passed, the *instructions* associated with that intent are generic. Whether the operator is performing a high-risk security audit or a low-risk documentation update, the system serves the same "Open Prompt" text. This forces the operator to manually "rehydrate" the specific rules of engagement (e.g., "Check for SQL injection," "Verify indentation") by typing them into the chat or pasting a separate file. This manual step is a vector for "Context Drift," where the actual instructions given to the AI diverge from the governance standards defined in the repository. Automating this via dynamic templates ensures that the context is rehydrated consistently, every time, using approved "Canon" definitions.

---

2. Governance Compliance Audit: The Policy of Truth

The design of any component within the Stela platform must undergo a rigorous compliance audit against the **Policy of Truth (PoT.md)**. The PoT is described as the "single constitutional document" and the "final authority" on all governance matters.[1] It is not a set of guidelines; it is a set of physical laws that the system must enforce.

2.1 The Filing Doctrine (Physical Law 1.1)

Section 1.1 of PoT.md defines the **Filing Doctrine**, which strictly partitions the repository based on file purpose [1]:

Directory	Purpose	Definition
ops/	**Run**	Binaries, manifests, automation. The machinery.
docs/	**Explain**	Manuals, rationale, prompt text. The instructions.
projects/	**Work**	Payload code.
storage/	**Trash**	Local artifacts, never canon.

Violation Analysis: The current implementation of ops/bin/open violates the Filing Doctrine. As evidenced by the source code in the dump [1], the binary contains a massive embedded heredoc (a block of text inside the script) that defines the "Stela OPEN PROMPT." This text includes instructions like "Adhere to the Behavioral Logic Standards defined in PoT.md" and "."

This architecture traps "Explanation" (the prompt text) inside "Run" (the binary). Ideally, ops/bin/open should contain *only* the logic to generate the prompt, not the content of the prompt itself. If an operator wishes to refine the wording of the "Role Contract," they currently have to edit the binary executable. This conflation of concerns increases the risk of accidental code breakage during text edits and makes the prompt text opaque to documentation tools that scan docs/.

Correction Strategy:

To adhere to the Filing Doctrine, the text of the prompts must be extracted from ops/bin/open and migrated to docs/ops/prompts/. The binary must then read these files at runtime. This restores the clean separation:

- **ops/bin/open:** Contains the logic to read a file, substitute variables, and write the output.
- **docs/ops/prompts/*.md:** Contains the actual text instructions for the AI.

2.2 The Single Source of Truth (SSOT) Axiom (Physical Law 1.2)

The **SSOT Axiom** states: "one canonical file per domain; other mentions are pointers".[1] In the context of prompt engineering, this means there must be one authoritative list of approved prompts. Allowing operators to pass arbitrary text files or untracked local snippets violates this axiom, creating "Shadow IT" or "Ghost Entities" (files that exist but are not governed).

Requirement: The implementation must introduce a formal registry, docs/ops/registry/PROMPTS.md, as the SSOT for all system prompts.[1] The ops/bin/open binary must strictly validate against this registry. It cannot simply load a file because it exists; it must confirm that the file is *registered*. This mechanism ensures that only "Canon" prompts—those that have been reviewed and committed to the repository—are used to initialize operational sessions.

2.3 Behavioral Logic Standards (Section 4.2)

PoT.md defines strict **Behavioral Logic Standards** that must be enforced by the system.[1]

- **Linguistic Precision:** The system must avoid ambiguity. A prompt intent of "check the code" is ambiguous. A prompt intent of "E-PROMPT-01" (Gatekeeper Audit) is precise. The new implementation must favor precise IDs over natural language descriptions in the CLI arguments.[1]
- **No Contractions:** The templates stored in docs/ops/prompts/ must adhere to the rule "No contractions across any scope".[1] The implementation of the templates must be linted to ensure "do not" is used instead of "don't," maintaining the formal, objective tone required by the governance layer.
- **Anti-Drift Governance:** "Logic or files misaligned with PoT.md are a system failure".[1] If the prompt customization system allows the injection of outdated or conflicting instructions, it facilitates drift. By centralizing prompts in docs/ and version-controlling them, the system ensures that the AI's instructions evolve synchronously with the codebase.

---

3. Current State Forensics: The Monolithic Barrier

To understand the necessity of the proposed changes, we must examine the limitations of the current ops/bin/open implementation. The source code provided in the dump [1] reveals a rigid, linear script.

3.1 Anatomy of the Current Binary

The script follows a predictable execution path:

1. **Argument Parsing:** It accepts flags like --intent, --dp, and --out.
2. **Git State Capture:** It executes git rev-parse to get the branch and hash.
3. **Static Generation:** It uses a cat <<EOF block to print a fixed string.

Bash

```
# Current Logic Snippet
{
  cat <<EOF
===== OPEN PROMPT =====
Stela OPEN PROMPT
...

- Active branch: $branch
- DP id/date (if applicable): $dp
...

- If any canon pointers are missing, say so.
===== END OPEN PROMPT =====
EOF
}
```

3.2 The Friction of Manual Fields

The user query highlights the workflow: **Copy/Paste Prompt -> Insert Fields**. In the current state, this "Insert Fields" step is a significant friction point.

When an operator runs ./ops/bin/open without extensive arguments, the output contains placeholders or empty fields:

- - DP id/date (if applicable): (none)
- - Intent for today: (often generic or empty)

The operator must then manually locate the Dispatch Packet (DP) ID—usually from TASK.md—and paste it into the prompt text before sending it to the AI. This manual transcription is a waste of human cognitive cycles and a potential source of error (e.g., typing "DP-OPS-0039" when the task is "DP-OPS-0038").

Furthermore, different tasks require different fields. A "Hygiene" task might require a target file path ({{TARGET}}), while an "Architect" task might require a summary plan (<summary-file>). The current monolithic prompt cannot adapt to these varying requirements,

forcing the operator to "invent" the structure manually, which violates the "No Invention" rule.[1]

3.3 The "Ghost Canon" Problem

The current script alludes to "Canon Pointers" but lists them statically:

- PoT.md
- TASK.md
- SoP.md

  ...

  This list is hardcoded. If the repository evolves and adds a new critical governance file (e.g., docs/SECURITY.md), the binary must be patched. Until it is patched, the prompt is providing stale pointers—a form of "Ghost Canon" where the definition of truth in the prompt lags behind the reality of the repo. By moving this list into editable Markdown templates in docs/, we decouple the *definition* of canon from the *execution* of the binary.

---

4. Target Architecture: The Context-Aware Template Engine

The proposed solution re-architects ops/bin/open from a static string emitter into a **Context-Aware Template Engine**. This engine has three primary components: the **Variable Harvesting Subsystem**, the **Registry Lookup Subsystem**, and the **Substitution Engine**.

4.1 The Prompt Registry (SSOT)

We establish docs/ops/registry/PROMPTS.md as the Single Source of Truth. Following the schema patterns seen in docs/ops/registry/AGENTS.md [1], this registry will define the authorized prompt configurations.

Table Schema:

The registry must use a valid Markdown table with specific columns to support automated parsing.

ID	Name	File Path	Variables	Notes
E-PROMPT-01	Gatekeeper	docs/ops/prompts/E-PROMPT-01.md	{{DP_ID}}, {{BRANCH}}	Audit stance.
E-PROMPT-0	Hygiene	docs/ops/prompts/E-PROMPT	{{DP_ID}},	Reformat/Clea

2		-02.md	{{TARGET}}	nup.
E-PROMPT-03	Architect	docs/ops/prompts/E-PROMPT-03.md	{{DP_ID}}, {{PLAN}}	Planning/Drafting.
E-PROMPT-04	Analyst	docs/ops/prompts/E-PROMPT-04.md	(none)	Read-only analysis.

Implications:

- **ID:** The CLI argument --intent=E-PROMPT-01 maps directly to this column.
- **File Path:** The binary uses this path to locate the template. If the path does not exist, the system fails (Dead End check).
- **Variables:** This column documents the specific data points the template expects, aiding in debugging and validation.

4.2 The Variable Schema and Harvesting Logic

To solve the "Insert Fields" friction, the system must automatically harvest data from the environment. The architecture defines a strict schema of substitution tokens.[1]

Variable Token	Source Logic	Purpose
{{DP_ID}}	Regex extraction from TASK.md header.	Identifies the active transaction.
{{BRANCH}}	git rev-parse --abbrev-ref HEAD	Physical state anchor.
{{HEAD}}	git rev-parse --short HEAD	Immutable time anchor.
{{ROOT}}	pwd	Context for relative paths.
{{TARGET}}	--target flag argument.	User-specified focus (e.g., specific file).
{{INTENT}}	--intent text (Legacy	Backwards compatibility.

	fallback).	

The Automation of {{DP_ID}}: Crucially, the {{DP_ID}} should not rely solely on user input. The TASK.md file is the "Active Work Surface".[1] Its header contains the authoritative DP ID (e.g., DP-OPS-0038). The binary must implement logic to read TASK.md, grep for the DP ID pattern (DP-[A-Z]+-[0-9]+), and auto-fill this variable. This eliminates the need for the operator to copy-paste the ID, directly supporting the "Zero Friction" requirement.

4.3 The Substitution Engine (Mechanism)

The engine is the Bash logic that replaces tokens in the template with harvested values.

Constraint: The implementation must use standard coreutils (sed, awk, grep) and avoid external dependencies like Python or Node.js to strictly adhere to the ops/ philosophy of being a lightweight kernel.[1]

Delimiter Risk and Mitigation:

A naive implementation using sed -e "s/{{BRANCH}}/$BRANCH/g" is dangerous. If a branch name contains a slash (e.g., work/refactor-auth), the sed command will crash because the slash conflicts with the substitution delimiter.

- **Solution 1:** Use a rare delimiter (e.g., | or #). sed -e "s|{{BRANCH}}|$BRANCH|g".
- **Solution 2:** Use native Bash string replacement (available in Bash 4+). ${template//\{\{BRANCH\}\}/$BRANCH}. This is safer and faster as it avoids spawning a subshell for sed.

Given the requirement for "Coding Best Practices," the native Bash replacement or a carefully escaped sed loop is preferred to ensure robustness against malformed input.

---

5. Implementation Strategy: Step-by-Step

This section outlines the precise changes required in ops/bin/open to implement the architecture.

5.1 Step 1: Formalizing the Registry

Before modifying the binary, the SSOT must be established.

- **Action:** Create docs/ops/registry/PROMPTS.md.
- **Content:** Populate it with the four canonical prompts defined in the specification.[1]
- **Validation:** Ensure tools/lint/library.sh (or a new linter) checks this registry for dead links.

5.2 Step 2: Template Migration

- **Action:** Extract the heredoc text from the current ops/bin/open [1] and save it to docs/ops/prompts/E-PROMPT-GENERIC.md.
- **Action:** Create docs/ops/prompts/E-PROMPT-01.md (Gatekeeper), 02.md (Hygiene), etc., extracting their specific requirements from the specifications.[1]
- **Syntax:** Replace static values with Handlebars tokens ({{DP_ID}}, {{BRANCH}}).

5.3 Step 3: Refactoring ops/bin/open

The binary will be rewritten to follow this pseudo-code logic:

1. **Initialize & Parse Args:**
   Parse --intent, --dp, --target, and --out.
2. **Harvest Context (The "Smart" Layer):**
   - Get GIT_BRANCH and GIT_HEAD.
   - Check TASK.md. Use grep -oE to find the DP-OPS-XXXX string.
   - Set DP_ID_VAR. Priority: 1) --dp flag, 2) TASK.md extraction, 3) (none).
3. **Resolve Template (The "Registry" Layer):**
   - Check if --intent matches an ID in docs/ops/registry/PROMPTS.md.
   - If Match: Extract the file path from the registry (Column 3). Verify file existence.
   - If No Match: Treat --intent as free text. Use docs/ops/prompts/E-PROMPT-GENERIC.md and pass the text to {{INTENT}}.
4. **Render Template (The "Engine" Layer):**
   - Read the resolved template file line by line.
   - Apply substitutions using safe delimiters or native Bash replacement.
   - Handle missing variables (e.g., if {{TARGET}} is needed but not provided, inject a visible placeholder like ``).
5. **Emit Output:**
   - Write the rendered content to storage/handoff/OPEN-....
   - Print the "OPEN saved" message.

5.4 Addressing the "Insert Fields" Workflow

The new logic directly attacks the "Insert Fields" bottleneck:

- **Current Workflow:** Run Open -> Output has empty fields -> User manually edits/fills fields.
- **New Workflow:** Run Open (System auto-harvests DP-ID from TASK.md) -> Output has fields pre-filled -> User verifies.

This transforms the user's role from data entry (high friction) to verification (low friction), achieving the requirement of supporting the operator's workflow while strictly adhering to PoT.md by ensuring the DP-ID comes from the authoritative source (TASK.md) rather than

human memory.

---

6. Detailed Analysis of Canonical Prompts

The specification identifies four specific prompts.[1] Each serves a distinct governance function and requires specific variable support.

6.1 E-PROMPT-01: Gatekeeper (Refresh + Audit)

- **Purpose:** To validate worker output before merging PRs.
- **Governance Requirement:** It must ensure the proofs match the allowlist defined in the DP.
- **Variables:**
  - {{DP_ID}}: Essential to link the audit to the specific request.
  - {{BRANCH}}: To ensure the audit is performed on the correct isolation branch.
- **Workflow Impact:** Operators use this when switching from "Contractor" (coding) mode to "Integrator" (review) mode.

6.2 E-PROMPT-02: Hygiene (Refresh + Conform DP)

- **Purpose:** To update an old or broken DP to the current TASK.md standard.
- **Governance Requirement:** "Drift Mitigation." It forces legacy artifacts to conform to new standards.
- **Variables:**
  - {{TARGET}}: The path to the specific Old-DP.md file being fixed.
- **Workflow Impact:** This prompt is unique because it targets a specific file. The --target flag in ops/bin/open is crucial here to populate the {{TARGET}} variable, avoiding the need for the operator to paste the file path manually.

6.3 E-PROMPT-03: Architect (Refresh + Draft DP)

- **Purpose:** Creating a new DP from a plan or conversation.
- **Governance Requirement:** "No Invention." The prompt explicitly instructs the AI to *draft* the DP based on a summary, strictly following the TASK.md schema.
- **Variables:**
  - {{DP_ID}}: The ID of the *new* task being created.
  - {{SUMMARY}} (or {{TARGET}}): The source material for the plan.

6.4 E-PROMPT-04: Analyst (Refresh + Discuss)

- **Purpose:** Analyzing the codebase without intent to edit.
- **Governance Requirement:** "Read-only mode." The prompt strictly forbids the generation of code or diffs.

- **Variables:** Minimal. Likely only {{BRANCH}} and {{HEAD}} to establish the context of the analysis.
- **Workflow Impact:** This is the "safe mode" for the AI, ensuring no accidental changes are suggested during exploration.

---

7. Security and Integrity Considerations

Implementing dynamic features introduces new attack surfaces or failure modes.

7.1 "Shadow IT" and Ghost Templates

Risk: An operator creates a local file my-prompt.md in storage/ that bypasses governance rules (e.g., instructing the AI to ignore PoT.md). **Mitigation:** The **Double-Check** mechanism.[1] ops/bin/open will *only* load files that are listed in docs/ops/registry/PROMPTS.md. Since the registry is a tracked file in git, adding a new prompt requires a Commit and a Pull Request, subjecting the new prompt to the standard repo-gates and human review.

7.2 Context Injection Hazards

Risk: A template includes a variable {{USER_INPUT}} that is populated directly from a CLI flag without sanitization. A malicious user could inject text that overrides the prompt's system instructions (e.g., Ignore previous instructions...).

Mitigation:

1. **Sanitization:** The ops/bin/open script should sanitize inputs (stripping control characters).
2. **Order of Operations:** The template structure should place user-injected variables *after* the core governance instructions (Physical Laws), ensuring that PoT.md directives take precedence in the AI's context window.

7.3 Delimiter Collision

Risk: As noted in section 4.3, simple sed substitution fails if the variable content includes the delimiter character.

Mitigation: Use native Bash substitution ${var//pattern/replacement}. This does not interpret the content as regex or delimiters, treating it as a literal string. This is the "Coding Best Practice" mandated by the user request.

Example of Robust Code:

Bash

```
# Safe substitution loop
while IFS= read -r line; do
  line="${line//\{\{DP_ID\}\}/$DP_ID}"
  line="${line//\{\{BRANCH\}\}/$BRANCH}"
  line="${line//\{\{TARGET\}\}/$ARG_TARGET}"
  echo "$line"
done < "$TEMPLATE_PATH"
```

---

8. Verification: The Linter

To ensure the system remains healthy, a verification layer must be added. The existing tools/verify.sh or tools/lint/truth.sh should be expanded, or a specific tools/lint/open.sh created.

Verification Logic:

1. **Registry Integrity:** Iterate through docs/ops/registry/PROMPTS.md. For every row, check if the file in Column 3 exists. Fail if missing (Dead Link).
2. **Reverse Integrity:** Iterate through docs/ops/prompts/*.md. Check if every file is listed in the Registry. Fail if not (Ghost Entity).
3. **Variable Check:** Scan templates for {{...}} tokens. Warn if a template uses a token that is not supported by the ops/bin/open logic (e.g., {{UNDEFINED_VAR}}), as this would leave ugly placeholders in the final prompt.

---

9. Conclusion

The transition of ops/bin/open to a **Context-Aware Template Engine** is a structural necessity for the Stela platform. It resolves the "Filing Doctrine" violations of the current monolithic binary, enforces the "SSOT Axiom" through a strict Registry, and significantly optimizes the operator's workflow by automating the "Insert Fields" phase via "Context Rehydration."

By implementing this architecture, the system moves from a passive tool that simply dumps text to an active participant in governance, ensuring that every AI session is initialized with a precise, version-controlled, and contextually accurate "Role Contract." This reduces the cognitive load on the operator, mitigates the risk of context drift, and hardens the repository against entropy.

The recommended path forward is to strictly follow the implementation steps: formalize the Registry, migrate the prompt content to docs/, and refactor the binary using the robust,

dependency-free Bash patterns detailed in this report. This ensures full compliance with PoT.md and prepares the platform for scalable, multi-modal AI operations.

Appendix: Data Tables

Table 1: Variable Substitution Schema

Variable	Description	Source Logic	Fallback
{{DP_ID}}	The active Dispatch Packet identifier.	Regex scan of TASK.md header (DP-[A-Z]+-[0-9]+) .	--dp flag or (none)
{{BRANCH}}	The current Git branch name.	git rev-parse --abbrev-ref HEAD	unknown
{{HEAD}}	Short hash of the current commit.	git rev-parse --short HEAD	unknown
{{ROOT}}	Repository root path.	pwd	.
{{TARGET}}	Specific file or target for the operation.	--target flag argument.	``
{{INTENT}}	Free-text intent (Legacy).	--intent flag argument.	Generic Prompt text.

Table 2: Filing Doctrine Compliance Matrix

Component	Current State	Location	Compliant?	Target State	Location	Compliant?
Binary Logic	Argument parsing, Git	ops/bin/open	Yes	Same + Template Engine.	ops/bin/open	Yes

	capture.					
Prompt Text	Hardcoded Heredoc string.	ops/bin/open	**NO**	Markdown Templates.	docs/ops/prompts/	**Yes**
Prompt List	Hardcoded if/else logic.	ops/bin/open	**NO**	Registry Table.	docs/ops/registry/	**Yes**
Context Logic	Manual user entry.	Operator Mind	**NO**	Automated Harvesting.	ops/bin/open	**Yes**

Works cited

1. dump-full-main-77413dda4.txt